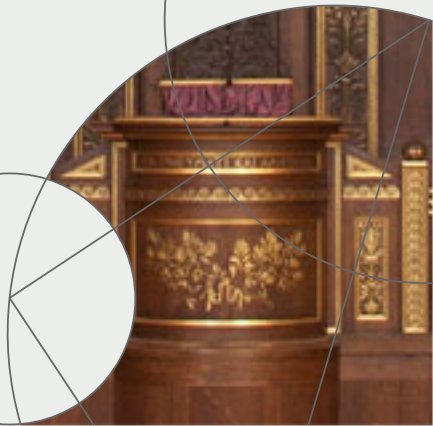
STEVENS INSTITUTE OF TECHNOLOGY

From Market Basket Analysis to Recommender

Rensheng Wang,
rwang1@stevens.edu
Dept. of Electrical and Computer Engineering
Stevens Institute of Technology



Market Basket Analysis

- To identify associations among items in transactional data - a practice commonly known as market basket analysis
- The result of a market basket analysis is a set of association rules that specify patterns of relationships among items
- A typical rule might be expressed in the form

$$\{\texttt{peanut butter}, \texttt{jelly}\} \quad \rightarrow \quad \{\texttt{bread}\}$$

- The association rule states that if peanut butter and jelly are purchased, then bread is also likely to be purchased.

The Apriori Algorithm for Association Rule Learning

- Transaction datasets are typically extremely large, both in terms of the # of transactions as well as the # of features.
- To reduce the number of itemsets to search, the most-widely used approach for efficiently searching large databases for rules is known as **Apriori**.
- The name is derived from the fact that the algorithm utilizes a simple prior (that is, a priori) belief about the properties of frequent itemsets.
- The Apriori algorithm employs a simple a priori belief as guideline for reducing the association rule search space: all subsets of a frequent itemset must also be frequent.
- This heuristic is known as the Apriori property. Using this astute observation, it is possible to dramatically limit the number of rules to search.
- For example, the set {`mobile oil`,`lipstick`} can only be frequent if both {`mobile oil`} and {`lipstick`} occur frequently as well. Consequently, if either motor oil or lipstick is infrequent, then any set containing these items can be excluded from the search.

The Apriori Algorithm for Association Rule Learning

- Strengths
  - ideally for very large transactional data
  - results in rules that are easy to understand
  - discovering unexpected knowledge in databases
- weaknesses
  - not very helpful for small datasets
  - takes effort to separate the insight from the common sense
  - easy to draw spurious conclusions from random patterns

The Apriori Algorithm for Association Rule Learning

☞ Consider a simple transaction database in a hospital gift shop for the principles

Transaction number	Purchased items
1	{flowers, get well card, soda }
2	{plush toy bear, flowers, balloons, candy bar}
3	{get well card, candy bar, flowers }
4	{plush toy bear, balloons, soda }
5	{flowers, get well card, soda }

The Apriori Algorithm for Association Rule Learning

- By looking at the sets of purchases, one can infer that there are a couple of typical buying patterns.
- A person visiting a sick friend or family member tends to buy a get well card and balloons, while visitors to new mothers tend to buy plush toy bears and balloons.
- Such patterns are notable because they appear frequently enough to explain the rule.
- In a similar fashion, the Apriori algorithm uses statistical measures of an itemset's "interestingness" to locate association rules in much larger transaction databases.
- Next, we will discover how Apriori computes such measures of interest, and how they are combined with the Apriori property to reduce the number of rules to be learned.

Measuring Rule Interest – Support and Confidence

- Whether or not an association rule is deemed interesting is determined by two statistical measures: **support** and **confidence.**
- By providing minimum thresholds for each of these metrics and applying the Apriori principle, it is easy to drastically limit the number of rules reported, perhaps even to the point where only the obvious, or common sense, rules are identified.
- Each of these metrics and applying the Apriori principle, it is easy to drastically limit the number of rules reported, perhaps even to the point where only the obvious, understand the types of rules that are excluded under these criteria.
- The support of an itemset or rule measures how frequently it occurs in the data. A function defining support for itemset X

$$\text{support}(X) = \frac{\text{count}(X)}{N}$$

where N is the number of transactions in the database and $\text{count}(X)$ indicates the number of transactions the itemset X appears in.

Support

Transaction number	Purchased items
1	{flowers, get well card, soda }
2	{plush toy bear, flowers, balloons, candy bar}
3	{get well card, candy bar, flowers }
4	{plush toy bear, balloons, soda }
5	{flowers, get well card, soda }

- For instance, the itemset {`getwellcard`,`flower`} has support of $3/5 = 0.6$ in the hospital gift shop data.
- Support can be calculated for any itemset, or even a single item; for example, support for {`candybar`} is 2/5 =0.4, since candy bars appear 40% of purchases.

Confidence

- Confidence is a measure of its predictive power or accuracy. It is defined as the support of the itemset containing both X and Y divided by the support of the itemset containing only X:

$$\text{confidence}(X \rightarrow Y) = \frac{\text{support}(X, Y)}{\text{support}(X)}$$

- Essentially, the confidence tells us the proportion of transactions where the presence of item or itemset X results in the presence of item or itemset Y.

☞ The confidence that $(X \rightarrow Y)$ is not the same as the confidence $(Y \rightarrow X)$.

- For example, the confidence of {`flowers`} → {`get well card`} is 0.6/0.8=0.75.
- In comparison, the confidence of $\{get\ well\ card\}$ → {`flowers`} is 0.6/0.6= 1.
- This means that a purchase involving flowers results is accompanied by a purchase of a great well card 75% of the time, while a purchase of a get well card is associated with flowers 100% of the time.
- This information could be quite useful to the gift shop management.

Support and Confidence

- The similarities between support, confidence, and the Bayesian probability rules.

$$\begin{aligned} \text{support}(A, B) &\Leftrightarrow P(A \cap B) \\ \text{confidence}(A \rightarrow B) &\Leftrightarrow P(B|A) \end{aligned}$$

- Rules like {`get well card`} → {`flowers`} are known as **strong rules** because they have both high support and high confidence.
- One way to find more strong rules would be to examine every possible combination of items in the gift shop, measure the support and confidence, and report back only those rules that meet certain levels of interest.
- However, this strategy is generally not feasible for anything but the smallest of datasets.

Building a Set of Rules with Apriori Principle

- Recall that the Apriori principle states that all subsets of a frequent itemset must also be frequent. In other words, if $\{A, B\}$ is frequent, then $\{A\}$ and $\{B\}$ both must be frequent.
- By definition, the support metric indicates how frequently an itemset appears in the data. Therefore, if we know that $\{A\}$ does not meet a desired support threshold, there is no reason to consider $\{A, B\}$ or any itemset containing $\{A\}$; it cannot possibly be frequent.
- The Apriori algorithms uses this logic to exclude potential association rules prior to actually evaluating them.
- The actual process of creating rules occurs in two phases:
  1. Identifying all item sets that meet a minimum support threshold
  2. Creating rules from these item sets that meet a minimum confidence threshold

Building a Set of Rules with Apriori Principle

- The first phase occurs in multiple iterations. Each successive iteration involves evaluating the support of storing a set of increasingly large itemsets.
- For instance, iteration 1 involves evaluating the set of 1-item itemsets, iteration 2 evaluates the 2-itemsets, and so on. The result of each iteration i is a set of all i-itemsets that meet the minimum support threshold.
- All the itemsets from iteration i are combined in order to generate candidate itemsets for evaluation in iteration $i+1$. But the Apriori principle can eliminate some of them even before the next round begins.
- For example, if $\{A\}$, $\{B\}$, $\{C\}$ are frequent in iteration 1 while $\{D\}$ is not frequent, then iteration 2 will consider only $\{A, B\}$, $\{A, C\}$, and $\{B, C\}$ since sets containing D have been eliminated a priori.
- Continuing this thought, suppose during iteration 2 it is discovered that $\{A, B\}$ and $\{B, C\}$ are frequent, but $\{A, C\}$ is not. Although iteration 3 would normally begin by evaluating the support for $\{A, B, C\}$, this step need not occur at all.

Building a Set of Rules with Apriori Principle

- Why not? The Aprior principle states that $\{A, B, C\}$ cannot possibly be frequent, since the subset $\{A, C\}$ is not.
- As having generated no new itemsets in iteration 3, the algorithm may stop Phase 1.
- The Phase 2 of Apriori algorithm may begin now. Given the set of frequent itemsets, association rules are generated from all possible subsets.
- For instance, $\{A, B\}$ would result in candidate rules for $\{A\} \rightarrow \{B\}$ and $\{B\} \rightarrow \{A\}$.
- These are evaluated against a minimum confidence threshold, and any rules that do not meet the desired confidence level are eliminated.

Identify Frequently Purchased Groceries with Association Rules

- We usually utilize data in the form of a matrix where rows indicated example instance and columns as features.
- In comparison, transactional data is more free-form. Each row in the data specifies a single transaction.
- However, each record comprises a comma-separated list of any number of items, from one to many.
- For instance, we have a sample file `groceries.csv` with the first 5 rows as below:

```
citrus fruit,semi-finishedbread,margarine,readysoups
tropical fruit,yogurt,coffee
whole milk
pip fruit,yogurt,cream cheese,meat spreads
other vegetables,whole milk,condensed milk,long life bakery product
```

Data Preparation: Create Sparse Matrix

- We introduce a data structure called a **sparse matrix**.
- Each row in the sparse matrix indicates a transaction.
- However, there is a column (feature) for every item that could possibly appear in someone's shopping bag.
- Say, if we have 169 different items in our grocery store, then our spares matrix will contain 169 columns.
- Unlike the conventional data frame or matrix, the sparse matrix does not actually store the zero (empty) values. This allows the structure to be more memory efficient.
- First to load the association rules package which can create the sparse matrix data structure.

```
> library(arules)
```

- The `read.transactions()` function can read the transaction file and generate a sparse matrix.

```
> groceries <-read.transactions("groceries.csv",sep=",")
```

Data Preparation: Create Sparse Matrix

- To see some basic information about the groceries dataset,

```
> summary(groceries)
transactions as itemMatrix in sparse format with
9835 rows(elements /itemsets /transactions) and
169 columns (items) and a density of 0.02609146
```

- The sparse matrix creates 9835 rows (i.e., 9835 transactions) and 169 columns which are features for each of the 169 different items in the grocery store.
- Each cell in the matrix is "**1**" if the item was purchased for the transaction, or "**0**" otherwise.
- The density value 0.02609146 (2.6%) refers to the proportion of non-zero matrix cells. On average, every transaction contains $169 \times 0.02609146 = 4.4095$ items.

Data Preparation: Create Sparse Matrix

- We can determine how often the items were purchased from the `summary()`

  most frequent items:

whole milk	other vegetables	rolls/buns	soda	yogurt	(Other)
2513	1903	1809	1715	1372	34055

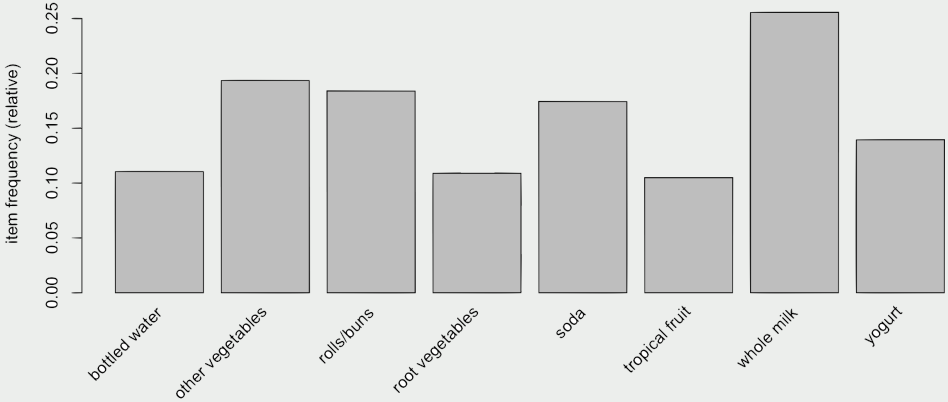
- Since 2513/9825=0.255, we can conclude that "whole milk" appeared 25.5% of transactions.

  element (itemset/transaction) length distribution:

size	1	2	3	4	5	6	7	8	9	10	11
freq.	2159	1643	1299	1005	855	645	545	438	350	246	182
size	12	13	14	15	16	17	18	19	20	21	22
freq.	117	78	77	55	46	29	14	14	9	11	4
size	23	24	26	27	28	29	32				
freq.	6	1	1	1	1	3	1				

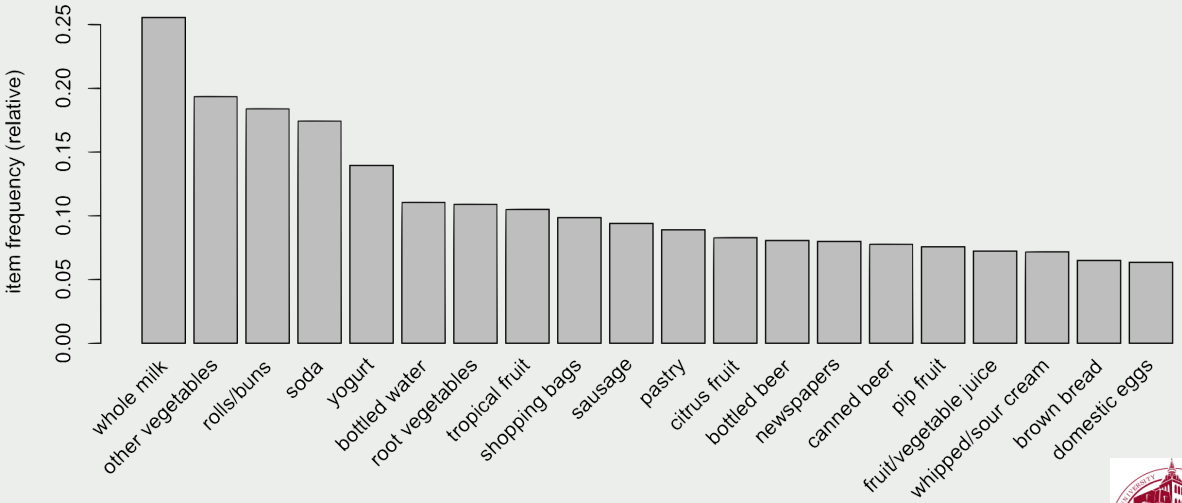
Top 10 Purchased Items

- `> itemFrequencyPlot(groceries,support = 0.1)`



Top 20 Purchased Items

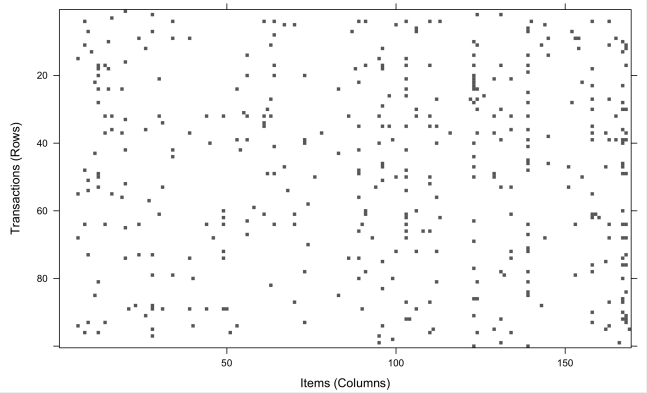
- `> itemFrequencyPlot(groceries,topN = 20)`



Visualizing Transaction Data

- `> image(sample(groceries,100))`

  This creates a matrix with 100 rows and the same 169 columns:



- A few columns seem fairly heavily populated, indicating some very popular items at the store.

Find Association Rules

- Use the `apriori()` function in the `arules` package to find the association rules
- Syntax

```
myrules <-apriori(data = groceries, parameter = list(support = 0.1,
                  confidence = 0.8, minlen = 1))
```

  - `groceries` is the sparse matrix holding 30 days transactional data
  - `support` specifies the minimum required rule support
  - `confidence` specifies the minimum required rule confidence
  - `minlen` specifies the minimum required rule items
- Examining Association Rules
- Syntax

```
inspect(myrules)
```

where `myrules` is a set of association rules from the `apriori()` function.

Choose Support & Confidence Parameter

- In order to produce a reasonable number of association rules, you need to choose a reasonable parameter with support and confidence levels.
- If you set these levels too high, then you might find no rules or rules that are too generic to be very useful.
- If the threshold is too low, you might end with an unwieldy number of rules, or the operations take too much time or memory during the learning phase.
- For example, for default `support =0.1`, this means that in order to generate a rule, an item must have appeared in at least 10% transactions. Since only 8 items appeared this frequently in our groceries data, it is no wonder we did not find any rules.
- One way to setup the support level is to think about the minimum number of transactions you would need before you could consider a pattern interesting.
- For instance, you could argue that if an item is purchased twice a day (about 60 times in 30 days), then it may be worth taking a look at. $60/9835 \approx 0.006$, i.e., `support`=0.006.

Choose Support & Confidence Parameter

- Setting the minimum confidence level involves a tricky balance.
- If the confidence is too low, then we might be overwhelmed with a large number of unreliable rules.
- If we set the confidence too high, then we will be limited to rules that are obvious or inevitable – like the fact that a smoke detector is always purchased in combination with batteries.
- Let us start with a confidence threshold of 0.25, which means that in order to be include in the results, the rule has to be correct at least 25% of the time. This will eliminate the most unreliable rules while allowing some room to modify behavior with targeted promotions.
- It is also helpful to set `minlen` = 2 to eliminate rules that contains fewer than two items.

```
myrules <- apriori(data = groceries, parameter = list(support = 0.006,
                   confidence = 0.25, minlen = 2))
```

- We obtain a set of 463 rules with the above command.

Evaluate Model Performance

- To obtain a high-level overview of the association rules, we can use `summary()` as follows.

  `summary(myrules)`

  rule length distribution: sizes

2	3	4
150	297	16

  where the size of rule is calculated as the total of both the left-hand side (lhs) and right-hand side (rhs) of the rule.

  This means that a rule like {bread} ⇒ {butter} is 2 items.

- We summarize statistics for the rule quality measures: `support`, `confidence` and `lift`.

- `lift` is a measure of how much more likely one item is to be purchased relative to its typical purchase rate, given that you know another item has been purchased. It is defined

$$\texttt{lift}(X \rightarrow Y) = \frac{\texttt{confidence}(X \rightarrow Y)}{\texttt{support}(Y)}$$

- Unlike confidence where the item order matters, $\texttt{lift}(X \rightarrow Y)$ is the same as $\texttt{lift}(Y \rightarrow X)$.

lift(X → Y)

- For example, suppose a grocery store, most people purchase milk and bread. By chance alone, we would expect to find many transactions with both milk and bread.
- However, if `lift (milk → bread)` is greater than 1, this implies that the two items are found together more often than one would expect by chance.
- A large lift value is therefore a strong indicator that a rule is important, and reflects a true connection between the items.
- We can take a look at specific rules using the `inspect()` function.

  `inspect(myrules[1:3])`

	LHS	RHS	support	confidence	lift
1	{pot plants}	⇒ {whole milk}	0.006914082	0.4000000	1.565460
2	{pasta}	⇒ {whole milk}	0.006100661	0.4054054	1.586614
3	{herbs}	⇒ {root vegetables}	0.007015760	0.4312500	3.956477

- The LHS is the condition that needs to be met in order to trigger the rule, and the RHS is the expected result of meeting that condition.

lift(X → Y)

- The first rule can be read as "if a customer buys potted plants, they will also buy whole milk." with `support` about 0.007 and `confidence` of 0.400, we can determine that this rule covers about 0.7% of transactions, and is correct in 40% of purchases involving potted plants.
- The `lift` value tells us how much more likely a customer is to buy whole milk relative to the average customer, given that he or she bought a potted plant.
- Since we know that about 25.6% of customers bought whole milk (the support) while 40% of customers buying a potted plant bought whole milk (the confidence), we can compute the lift as $0.40/0.256 = 1.56$, which matches the value shown.
- Note that here the support indicates the support for the rule, not the support for the lhs or rhs.

Evaluate Rules

- A common approach is to take the result of learning association rules and divide them into 3 categories:
  - Actionable
  - Trivial
  - Inexplicable
- The goal of market basket analysis is to find **actionable** associations, or rules that provide a clear and useful insight.
- **Trivial** rules include any rules that are so obvious that they are not worth mentioning – they are clear, but not useful. Like `{diapers}` $\Rightarrow$ `{formula}`
- Rules are **inexplicable** if the connection between the items is so unclear that figuring out how to use the information for action would require additional research.
- The best rules are the hidden gems – those undiscovered insights into patterns that seem obvious once discovered.

Sorting the Association Rules

- The most useful rules might be those with the highest support, confidence, or lift.
- To reorder the set of rules, we can apply `sort()` while specifying a by parameter of `support`, `confidence`, and `lift`
- To find the best 5 rules according to the lift statistic:

  `inspect(sort(myrules, by = "lift")[1:5])`

	LHS	RHS	support	confidence	lift
1	{herbs}	⇒ {root vegetables}	0.00701	0.43125	3.9564
2	{berries}	⇒ {whipped/sour cream}	0.00904	0.27217	3.7968
3	{tropical fruit, other vegetables, whole milk}	⇒ {root vegetables}	0.00701	0.41071	3.7680
4	{beef, other vegetables}	⇒ {root vegetables}	0.00793	0.40206	3.6886
5	{other vegetables, tropical fruit}	⇒ {pip fruit}	0.00945	0.26345	3.4826

Sorting the Association Rules

- To find the best 5 rules according to the lift statistic:

  `inspect(sort(myrules, by = "lift")[1:5])`

	LHS	RHS	support	confidence	lift
1	{herbs}	⇒ {root vegetables}	0.00701	0.43125	3.9564
2	{berries}	⇒ {whipped/sour cream}	0.00904	0.27217	3.7968
3	{tropical fruit, other vegetables, whole milk}	⇒ {root vegetables}	0.00701	0.41071	3.7680
4	{beef, other vegetables}	⇒ {root vegetables}	0.00793	0.40206	3.6886
5	{other vegetables, tropical fruit}	⇒ {pip fruit}	0.00945	0.26345	3.4826

- The 1st rule, with a lift of 3.95664, implies that people who buy herbs are nearly 4 times more likely to buy root vegetables than typical customers.

Take Subset of Association Rules

- Sometimes, the marketing team is wondering whether some certain items are often purchased with other items, say berries.
- We need to find all the rules that include berries in some form.
- The `subset()` function provides a method for searching for subset of transactions, items, or rules.

  `berryrules <-subset(myrules,items%in%"berries")`
- The `subset()` function can be used with several keywords and operators:
  - The keyword `items`, matches an item appearing anywhere in the rule
  - The operator `%in%` means that at least one of the items must be found in the list, say, `items%in%c("berries","yogurt")`
  - Partial matching use `%pin%` and complete matching `%ain%`
  - Subset can also be limited by support, confidence, or lift. Say, `confidence > 0.50` will limit to rules with confidence >50%